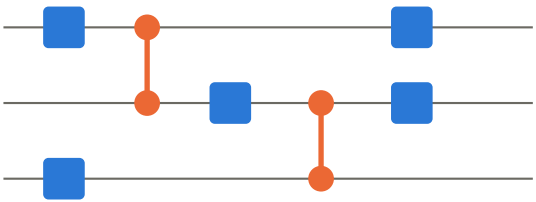
PROGRESS UPDATE

Making quantum circuits shorter, and proving how short they can get

What the original paper does, what we added, and what the numbers say

Arnav Kapoor · IISER Bhopal · qcr extension paper · October 2026

Every gate costs reliability



A circuit is a list of **gates** on **qubits** (the lines). **Blue** gates touch one qubit. **Orange** gates link two qubits and are about ten times more error-prone.

100 gates at 99% each

37%

chance of no error

same job in 50 gates

61%

shorter means better answers

This product of gate fidelities is the ESP (estimated success probability), the score we report on hardware.

The original method: swap a piece for a shorter one

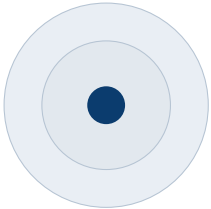
STEP 1
Pick a window
2 to 8 gates in a row

→

STEP 2
Look it up
a shorter circuit
that does the same

→

STEP 3
Replace, repeat
until nothing shrinks



radius r

The lookup table is a map of short circuits.

Start from "do nothing" (the dot) and add one gate at a time. Every circuit reachable in at most r steps is stored with its shortest form. This map is the **compute graph**, and r is the **database radius**. The original paper uses $r = 5$ or 6.

Rosenhahn, Osborne and Hirche, New J. Phys. 27, 104509 (2025). Variants V1 (random search), V2 (database), V3 (database with a random forest).

TWO QUESTIONS THE ORIGINAL PAPER LEAVES OPEN

1

How far from the best possible is it?

They only compare with Qiskit, so a 45-gate result cannot be judged. Is the best 40 gates, or 10?

2

Does it help on circuits people actually run?

Their tests use random circuits, a few fixed gates, every qubit linked to every other, and only count gates.

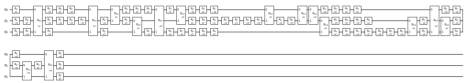
Question 1: we computed the true shortest circuit

Their gates on 3 qubits only make a finite set of operations. We listed **all 92,897,280** of them and stored the exact shortest length of each.

So for any test circuit we know the **best possible answer**, not just "better than Qiskit".

Beyond this set (Clifford+T gates) we do the same for 2 qubits, and hide a known short core inside long circuits.

(a) input: random 3-qubit word over the ion alphabet, 80 gates



(b) original method (database radius 5): 24 gates

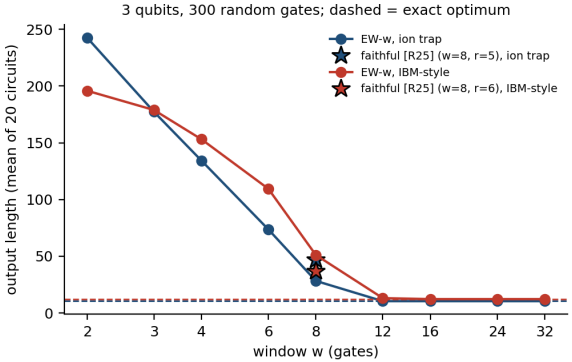


(c) proven optimum (exact Cayley table): 11 gates



Example: an 80-gate random circuit. The original method stops at 24 gates. The proven optimum is 11.

Why it falls short: the lookup table is too small



Two knobs decide the result. Both must be large enough.

Window $w \geq 12$ to 16 gates
local fixes then add up to the global optimum (we prove when)

Table radius $r \geq 8$
the original uses 5 or 6, and this is what holds it back

3 qubits, 300 random gates, 20 circuits. Stars: the original method. Dashed: the proven optimum.

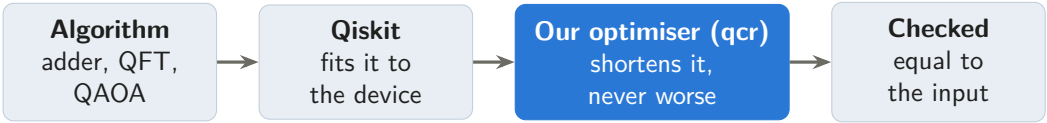
On their own tests, we land on the optimum

Test (mean gates, 300 in)	Qiskit	Original	Ours	Best possible
3 qubits, ion gates	127	47.1	**10.8**	10.4
3 qubits, IBM-style gates	104	36.8	**12.5**	12.2
4 qubits, ion gates	165	78.8	**46.1**	at least 11
4 qubits, IBM-style gates	141	76.0	**51.4**	at least 13
15 qubits (500 in)	332	336	**328**	unknown

3 qubits: ours is **optimal on 35 of 40** circuits. The original is 3 to 4.5 times too long.

Clifford+T, 2 qubits: ours is **optimal on 99/99** random and 41/41 planted circuits (original 81/99).

Question 2: our optimiser, built for real devices



Bigger tables, smarter windows
deep hashed tables, windows that skip gates which do not interact

Uses the device's error rates
removes the gates that hurt most, not just the most gates

Exact, down to the phase
output equals input exactly, no approximation

Any gate set, any device
no group theory inside, tested on IBM Heron and trapped ions

60 real algorithm circuits on IBM Heron

Compared with Qiskit level 3	2-qubit	all gates	ESP
Qiskit level 3	1.00	1.00	1.00
TKET	1.02	1.71	1.16
Ours, after level 3	**0.98**	**0.87**	**1.03**
Ours, after level 1	**1.00**	**0.84**	**1.21**
BQSKit level 3 (approx.)	0.85	1.20	1.22
Ours, after BQSKit	**0.85**	**1.01**	**1.25**

Counts: **lower is better**.
ESP: **higher is better**.

After Qiskit, ours **never adds** a 2-qubit gate and cuts 13% of all gates.

BQSKit finds better 2-qubit structure but leaves many 1-qubit gates. Run ours after it and you get both: its 2-qubit savings and 16% fewer gates, still exact.

58 circuits compared, 2 to 12 qubits. Trapped-ion results point the same way, with smaller margins.

It scales: circuits of 14 to 100 qubits

19%
fewer gates than Qiskit level 3 alone, on IBM Heron

20/22
circuits lose 2-qubit gates, none gains one

95
qubits in the largest compared circuit, up to 5,425 input gates

The cost: time. Minutes per circuit, up to 6 hours for the biggest, in single-threaded Python. Qiskit takes under a second.

Still running. BQSKit on this suite (up to 40 qubits), and 2 of 90 large runs. Numbers here are from 88 runs.

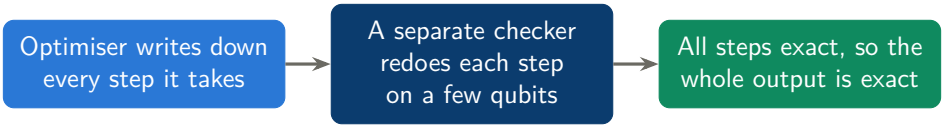
45 Nam/VOQC circuits. Product-formula circuits excluded, as Qiskit drops their tiny rotations approximately.

How we know every output is correct

UP TO 24 QUBITS
Simulate both
on random states and compare. Largest difference: $1.4 \cdot 10^{-10}$

ANY SIZE
Replay certificate
simulation is impossible past about 25 qubits, so we check the steps

CLIFFORD GATES
Exact arithmetic
no rounding at all, equal or not equal



Large suite: 171 of 176 outputs certified, each check under 7 seconds. The other 5 hit an old checker limit and are being re-run.

What is left before submission

Item	Status
BQSKit on the large suite, 5 certificate re-checks	running
Fairer ESP reporting for very large circuits	done
More baselines (VOQC, Quartz, QUESO)	next
Several random seeds, with error bars	next
Ablation: which part of the optimiser matters	next
Runs on real IBM hardware	last, needs access

In one line: the original method is 3 to 4.5 times off the optimum, we can prove it, and a general exact optimiser closes that gap and still helps on real circuits up to 95 qubits.